

Terraform Commands (Basic to Advanced) – Hinglish Deep Guide

Yeh document Terraform ke sabhi important commands ko **basic se advanced DevOps level tak Hinglish mein explain karta hai**, with real-world reasoning.

1. terraform init (Project Setup)

Kya karta hai?

Terraform project ko initialize karta hai.

Inside kya hota hai:

- `.terraform/` folder create hota hai
- required providers download hote hain (AWS, Azure, etc.)
- backend setup hota hai (S3, Terraform Cloud)
- modules fetch hote hain

DevOps Reasoning:

Terraform engine ko start karna = dependencies install + environment ready

Important:

Bina init ke `plan/apply` possible nahi hota.

2. terraform plan (Dry Run)

Kya karta hai?

Batata hai kya changes honge AWS infra me.

Inside process:

- current state file read hota hai
- AWS real infra se compare hota hai
- diff generate hota hai

DevOps Reasoning:

Ye infrastructure ka "Git diff" hai

👉 Actions:

- ◦ create
 - ~ modify
 - ◦ destroy
-

3. terraform apply (Execution)

👉 Kya karta hai?

Actual infrastructure create/update/delete karta hai.

 Inside flow:

1. plan generate
2. user approval
3. AWS API calls execute
4. state file update

 Reasoning:

Apply = production change execution step

⚠️ Risk: wrong apply = infra damage

4. terraform destroy (Infra Delete)

👉 Kya karta hai?

Saara infra delete karta hai.

 Inside:

- state file read hota hai
- reverse dependency order me delete hota hai



Use Case:

- dev environment cleanup
- cost saving



Production me dangerous command



5. terraform validate (Code Check)



Kya karta hai?

Terraform code syntax validate karta hai.



Reasoning:

Compile-time check jaisa hai



AWS infra validate nahi karta



6. terraform fmt (Code Formatter)



Kya karta hai?

Code ko standard format me convert karta hai.



Reasoning:

Team consistency maintain karna



7. terraform show (State Viewer)



Kya karta hai?

State file ko readable format me show karta hai.



Inside:

.tfstate decode hota hai

8. terraform output

Kya karta hai?

Output variables show karta hai.

Use:

- EC2 IP
- LB DNS
- pipeline integration

9. terraform state commands (VERY IMPORTANT)

state list

All resources list karta hai

state show

Specific resource details show karta hai

state mv

Resource ko rename/move karta hai

 Important: Terraform ko batana padta hai ki resource same hai warna recreate ho jata hai

state rm

State se resource remove karta hai (AWS se nahi)

10. terraform import (Legacy Infra)

Kya karta hai?

Existing AWS infra ko Terraform me bring karta hai.

Use Case:

- legacy infra onboarding
- brownfield migration

```
terraform import aws_instance.myec2 i-123456
```

11. terraform taint (Force Recreate)

Kya karta hai?

Resource ko mark karta hai for recreation.

 Next apply = destroy + recreate

Use:

- corrupted instance
- config issue

12. terraform untaint

 taint ko remove karta hai

13. terraform workspace (Multi Environment)

Kya karta hai?

Same code multiple environments me run karta hai.

Example:

- dev
- stage
- prod

Inside:

Same infra code, different state files

14. terraform force-unlock

Kya karta hai?

Stuck state lock ko unlock karta hai.

Reason:

- crash during apply
 - lock not released
-

15. terraform plan/apply with file

Save plan:

```
terraform plan -out=tfplan
```

Apply same plan:

```
terraform apply tfplan
```

Reason:

CI/CD consistency ensure karta hai



16. Debugging (TF_LOG)

```
TF_LOG=DEBUG terraform apply
```

Levels:

- TRACE (deep internal)
 - DEBUG (API calls)
 - INFO
 - ERROR
-



REAL WORLD FLOW

```
terraform init
terraform validate
terraform fmt
terraform plan -out=tfplan
terraform apply tfplan
```



FINAL MINDSET

Terraform = Desired State Engine

👉 Tu bolta hai "kya chahiye" 👉 Terraform ensure karta hai "kaise milega"



INTERVIEW LINE

Terraform = Infrastructure as Code + State Management + Drift Detection + Desired State Reconciliation